

FIAP GRADUAÇÃO

TECNOLOGIA EM DESENVOLVIMENTO DE SISTEMAS

DevOps Tools & Cloud Computing

Azure Container Instances (ACI) e Azure Container Register (ACR)

PROF. João Menk

profjoao.menk@fiap.com.br

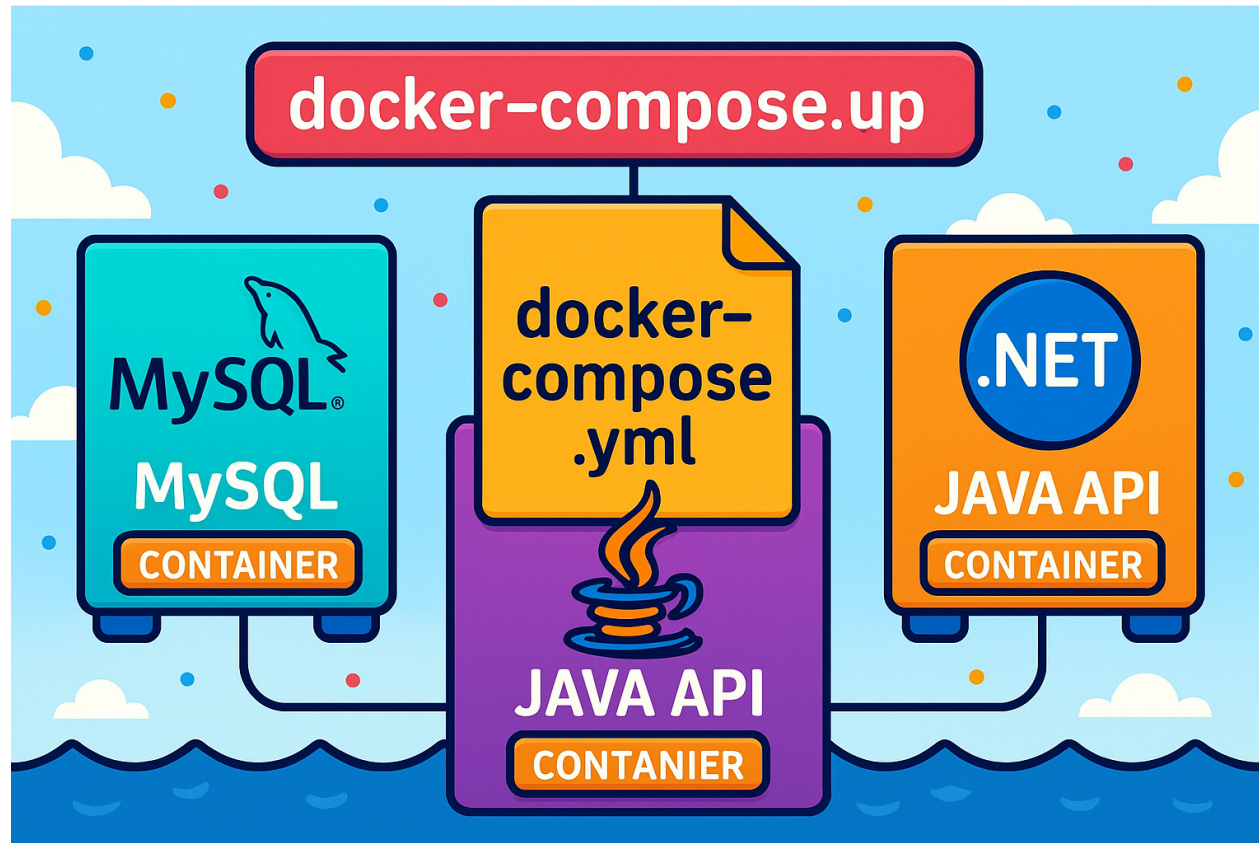
AZURE CONTAINERS TUTORIAL

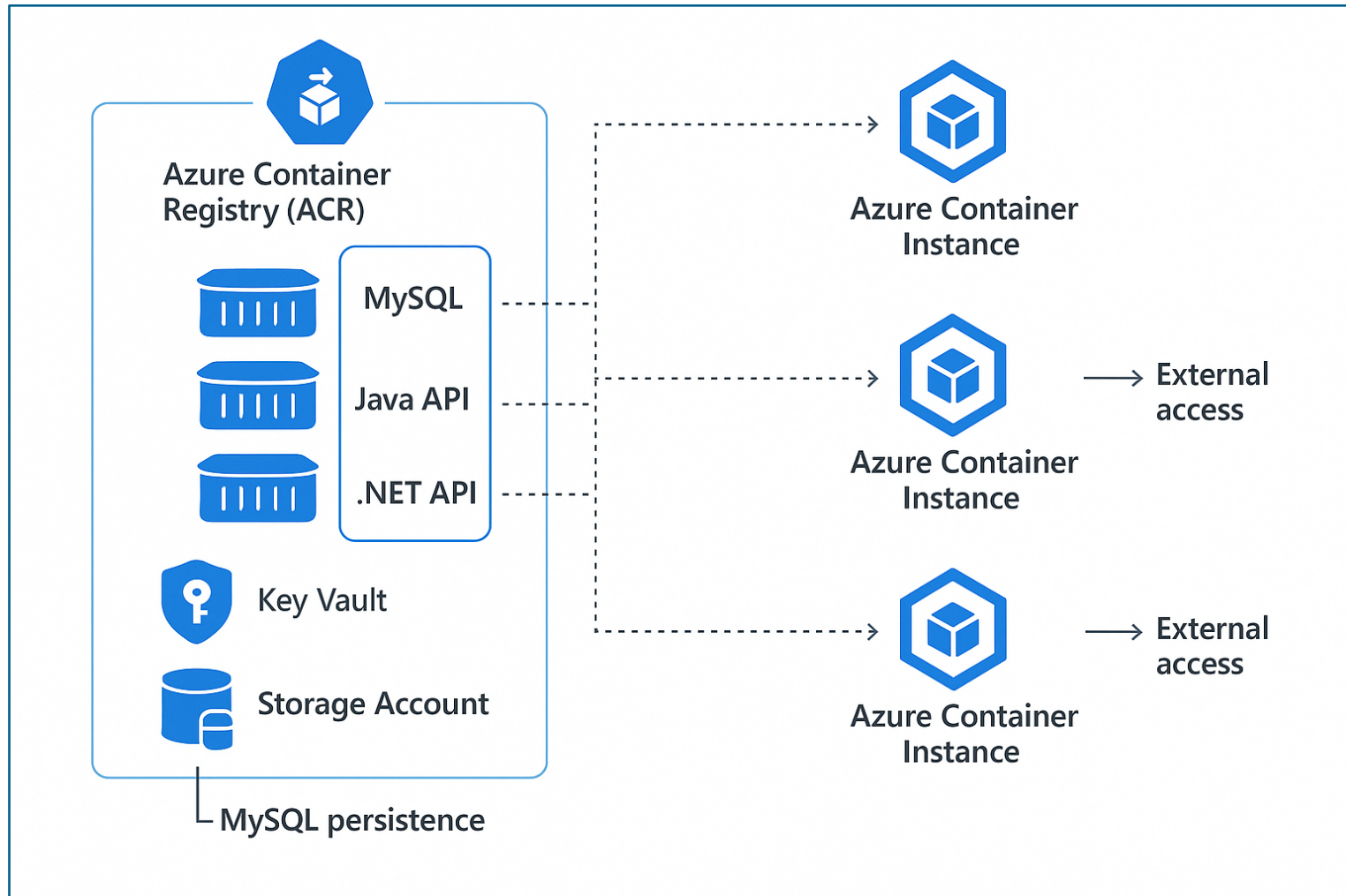


DimDim

Cenário: Sistema de Transações Bancárias DimDim

Vamos resgatar nosso último exercício realizado com Docker Compose para aprender na prática o funcionamento do ACR e ACI





Clone dos Projetos

Vamos começar a tarefa clonando os projetos do GitHub

Na HOME do seu usuário

```
[admlnx@linux-free ~]$ pwd  
/home/admlnx
```



`git clone https://github.com/profjoaomenk/mysql-dimdim-2.git`



`git clone https://github.com/profjoaomenk/transacoes-api-2.git`



`git clone https://github.com/profjoaomenk/ApiTransacoes-2.git`

Agora vamos criar as imagens dos nossos serviços

Entre no diretório **mysql-dimdim-2**



```
docker build -f Dockerfile.mysql -t mysql-dimdim .
```

Entre no diretório **transacoes-api-2**



```
docker build -f Dockerfile.api -t api-dimdim .
```

Entre no diretório **ApiTransacoes-2**



```
docker build -f docker/Dockerfile -t api-transacoes .
```



Máquina Virtual Linux COMPUTAÇÃO

750 horas

cada uma das VMs com capacidade de intermitência B1s e B2ats v2 (baseadas em AMD).

Crie VMs (máquinas virtuais) do Linux em segundos para atender às suas necessidades de carga de trabalho e orçamento. [Saiba mais](#)

Criar



Registro do Contêiner CONTÊINERES

1 Registro de camada padrão

Com armazenamento de 100 GB e 10 webhooks

Armazenar e gerenciar imagens de contêiner em todos os tipos de implantações do Azure. [Saiba mais](#)

Criar



Arquivos do Azure ARMAZENAMENTO

100GB

de transações LRS otimizadas, k arquivos quentes e frios. 2 milhões de operações de leitura, lista e outras operações de arquivo

Migre para o armazenamento de arquivos simples, distribuído, de plataforma cruzada sem alterar o código. [Saiba mais](#)

Criar



Key Vault SEGURANÇA

10.000 transações

Chaves RSA de 2048 bits ou operações secretas, Camada padrão.

Proteja e mantenha o controle sobre chaves e outros segredos. [Saiba mais](#)

Criar



Azure Container Registry



Um Registro

- Um registro de Container é um serviço que armazena e distribui imagens de Container e artefatos relacionados
- O Docker Hub é um exemplo de um registro de Container público que serve como um catálogo geral de imagens de Container do Docker
- O Registro de Container do Azure fornece aos usuários o controle direto de seu conteúdo de Container, com **autenticação integrada**, **replicação geográfica** que dá suporte à distribuição global e confiabilidade para implantações, configuração de rede virtual com **Link Privado** e muitos outros recursos aprimorados. Oferece também a utilização em conjunto com outros serviços do Azure, como por exemplo Web App (Serviços de Aplicativos)



Vários Repositórios

- Um repositório é uma **coleção de imagens** de Container ou de outros artefatos **em um registro** que têm o mesmo nome, **mas labels diferentes**. Por exemplo, as três imagens a seguir estão no [repositório acr-dimdim](#):

acr-dimdim:latest

acr-dimdim:v1

acr-dimdim:v2

- Podemos também incluir nomes de repositório (namespaces). Os namespaces permitem que você identifique os repositórios relacionados e a propriedade do artefato em sua organização usando nomes delimitados por barra. Por exemplo:

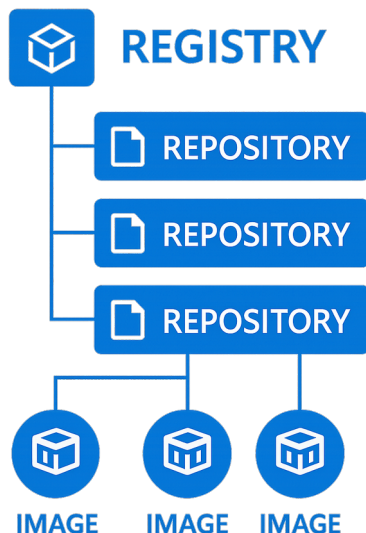
marketing/campanha10-18/web:v2

marketing/campanha10-18/api:v3



Várias Imagens

- Todas as imagens de Container e de outros artefatos no registro são **criptografadas em repouso**. O Azure criptografa automaticamente uma imagem antes de armazená-la e a descriptografa dinamicamente quando você ou seus aplicativos e serviços obtêm a imagem
- Armazena dados na **região em que o registro é criado**, para ajudar os clientes a atender aos requisitos de conformidade e residência de dados



Criar um Registro de Container Privado usando a CLI do Azure

- O Azure Cloud Shell não inclui todos os componentes de Docker necessários (o daemon docker), por isso **não podemos utilizar o Cloud Shell** para este exercício
- Como ferramentas iremos precisar do **CLI do Azure** e o **Docker** instalado
- Vamos utilizar nossa VM Linux com Docker ou você pode utilizar seu próprio equipamento com as ferramentas instaladas

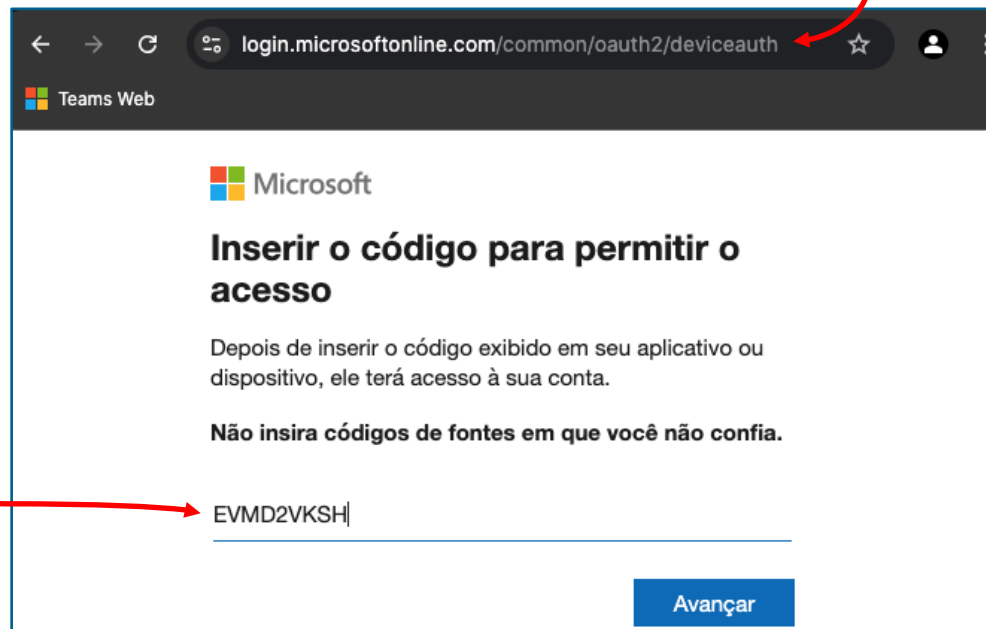


Criar um Registro de Container Privado usando a CLI do Azure

1. Abra seu terminal e realize o login com sua conta da Azure

az login

```
Menk — admInx@linux-free:~ — ssh admInx@104.41.52.221 — 95x24
[admInx@linux-free ~]$ az login
To sign in, use a web browser to open the page https://microsoft.com/devicelogin and enter the
code EVMD2VKSH to authenticate.
```



Criar um Registro de Container Privado usando a CLI do Azure

Após realizar a autenticação escolha sua assinatura

```
[admlnx@linux-free ~]$ az login
To sign in, use a web browser to open the page https://microsoft.com/devicelogin and enter the code EVMD2VKSH to au
thenticate.

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No      Subscription name                Subscription ID                Tenant
-----
[1] *   pf0841g1 - Azure para Estudantes  f85ce35b-59c2-4fb8-9627-68c4aa18fcce  Fiap-Faculdade de Informática e Admi
nistração Paulista
[2]     pf0841s1 - Azure para Estudantes  ed71109f-42b0-4088-89d7-41d7ec8217cc  Fiap-Faculdade de Informática e Admi
nistração Paulista

The default is marked with an *; the default tenant is 'Fiap-Faculdade de Informática e Administração Paulista' and
subscription is 'pf0841g1 - Azure para Estudantes' (f85ce35b-59c2-4fb8-9627-68c4aa18fcce).

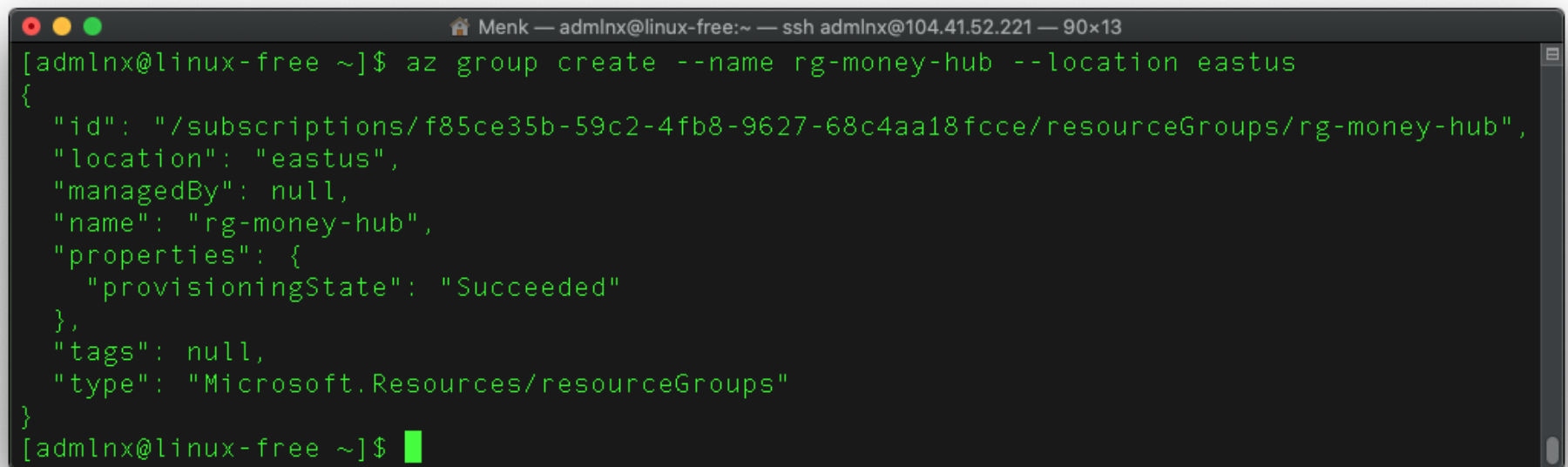
Select a subscription and tenant (Type a number or Enter for no changes): 1
```

az account show

Criar um Registro de Container Privado usando a CLI do Azure

2. Crie um Grupo de Recursos para abrigar os recursos que serão criados

az group create --name rg-money-hub --location eastus



```
Menk — admlnx@linux-free:~ — ssh admlnx@104.41.52.221 — 90x13
[admlnx@linux-free ~]$ az group create --name rg-money-hub --location eastus
{
  "id": "/subscriptions/f85ce35b-59c2-4fb8-9627-68c4aa18fcce/resourceGroups/rg-money-hub",
  "location": "eastus",
  "managedBy": null,
  "name": "rg-money-hub",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
[admlnx@linux-free ~]$
```

(AuthorizationFailed) The client 'pf0841g1@fiap.com.br' with object id

az account list -o table

az account set --subscription <"nome" ou id>

Criar um Registro de Container Privado usando a CLI do Azure

3. Agora vamos criar um Registro de Container **Privado**

O nome do registro deve ser único no Azure e conter de 5 a 50 caracteres alfanuméricos (preferencialmente em minúsculas)

```
az provider register --namespace Microsoft.ContainerRegistry
```

```
az acr create \  
  --resource-group rg-money-hub \  
  --name moneyhubrm9999 \  
  --sku Standard \  
  --location eastus \  
  --public-network-enabled true \  
  --admin-enabled true
```

Altere para seu RM

Navegue no Portal da Azure juntamente com o Professor para analisar o Registro de Container criado

Registro de Container no Azure



Criar um Registro de Container Privado usando a CLI do Azure

4. Vamos recuperar algumas informações preciosas do nosso Registro de Container

```
LOGIN_SERVER=$(az acr show --name moneyhubrm9999 \
                    --resource-group rg-money-hub \
                    --query loginServer --output tsv)

echo ""
echo "Login Server: $LOGIN_SERVER"
echo ""
```

```
ADMIN_USERNAME=$(az acr credential show --name moneyhubrm9999 \
                    --resource-group rg-money-hub \
                    --query username --output tsv) && \
ADMIN_PASSWORD=$(az acr credential show --name moneyhubrm9999 \
                    --resource-group rg-money-hub \
                    --query passwords[0].value --output tsv) && \
echo "Username: $ADMIN_USERNAME" && echo "Password: $ADMIN_PASSWORD"
```

Conseguimos essas informações no Portal também

Criar um Registro de Container Privado usando a CLI do Azure

5. Antes de efetuar Push e Pull das imagens, precisamos fazer o login no Registro
Existem duas maneiras (*ambas precisam do Docker instalado*):

az acr login: Usa autenticação via Azure AD (*azure login já efetuado*), não recebe senha

docker login: Aceita usuário e senha explicitamente

az acr login --name moneyhubrm9999

**docker login moneyhubrm9999.azurecr.io **
**-u \$ADMIN_USERNAME **
-p \$ADMIN_PASSWORD



Criar um Registro de Container Privado usando a CLI do Azure

Para realizar o Login sem o Docker estar instalado, utilize os comandos abaixo, após estar Logado na conta da Azure (**az login**)

```
az acr login --name moneyhubrm9999 --expose-token
```

```
TOKEN=$(az acr login --name moneyhubrm9999 --expose-token --output tsv --query accessToken)
```

```
echo $TOKEN | docker login moneyhubrm9999.azurecr.io --username 00000000-0000-0000-0000-000000000000 --password-stdin
```

Criar um Registro de Container Privado usando a CLI do Azure

6. Vamos subir as nossas imagens agora. Precisamos remarcar essas imagens utilizando o comando **docker tag**, criando assim uma nova imagem com um **namespace** padronizado para subir na Azure

docker image ls

```
[admlnx@linux-free ~]$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
api-transacoes	latest	47a9f310bb5c	2 hours ago	274MB
api-dimdim	latest	b2d7d3895b76	2 hours ago	358MB
mysql-dimdim	latest	66f4273bcc43	2 hours ago	781MB

docker tag mysql-dimdim moneyhubrm9999.azurecr.io/mysql-dimdim:v1

Nome da imagem origem

Registro

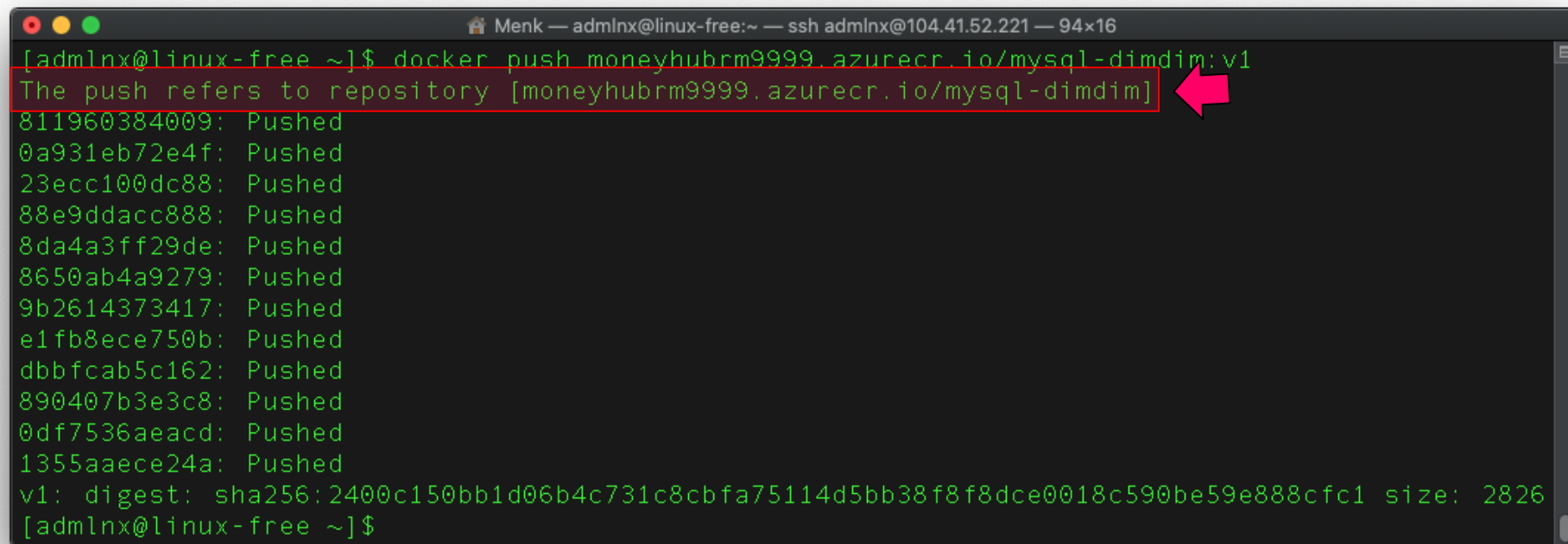
Repositório

Imagem

Criar um Registro de Container Privado usando a CLI do Azure

7. Podemos agora realizar o Push dessa imagem para o Registro de Container no Azure

docker push moneyhubrm9999.azurecr.io/mysql-dimdim:v1



```
Menk — admlnx@linux-free:~ — ssh admlnx@104.41.52.221 — 94x16
[admlnx@linux-free ~]$ docker push moneyhubrm9999.azurecr.io/mysql-dimdim:v1
The push refers to repository [moneyhubrm9999.azurecr.io/mysql-dimdim]
811960384009: Pushed
0a931eb72e4f: Pushed
23ecc100dc88: Pushed
88e9ddacc888: Pushed
8da4a3ff29de: Pushed
8650ab4a9279: Pushed
9b2614373417: Pushed
e1fb8ece750b: Pushed
dbbfcab5c162: Pushed
890407b3e3c8: Pushed
0df7536aeacd: Pushed
1355aaece24a: Pushed
v1: digest: sha256:2400c150bb1d06b4c731c8cbfa75114d5bb38f8f8dce0018c590be59e888cfc1 size: 2826
[admlnx@linux-free ~]$
```

Criar um Registro de Container Privado usando a CLI do Azure

8. Vamos remarcar as outras imagens que temos e subir para o ACR também

`docker tag` api-dimdim moneyhubrm9999.azurecr.io/api-dimdim:v1

Nome da imagem origem Registro Repositório Imagem

`docker push` **moneyhubrm9999.azurecr.io/api-dimdim:v1**

`docker tag` api-transacoes moneyhubrm9999.azurecr.io/api-transacoes:v1

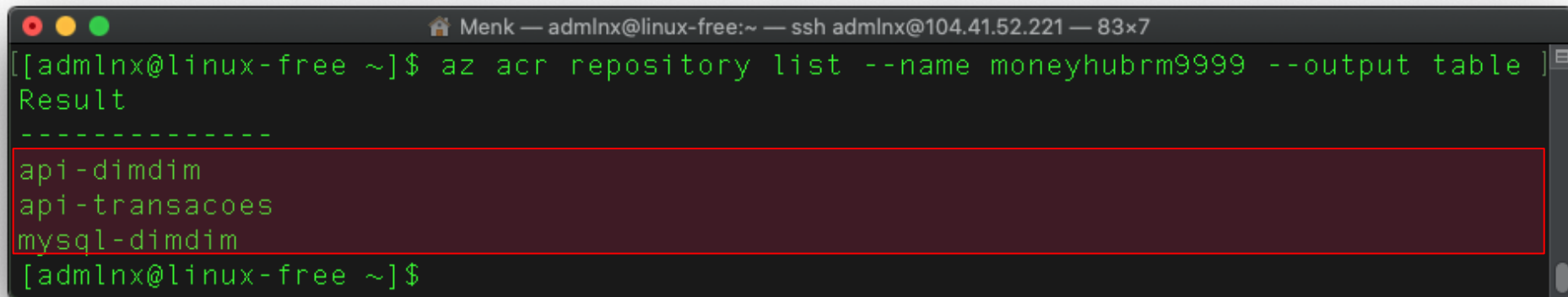
Nome da imagem origem Registro Repositório Imagem

`docker push` **moneyhubrm9999.azurecr.io/api-transacoes:v1**

Criar um Registro de Container Privado usando a CLI do Azure

9. Vamos listar as imagens que possuímos em nosso registro

az acr repository list --name moneyhubrm9999 --output table



```
Menk — admlnx@linux-free:~ — ssh admlnx@104.41.52.221 — 83x7
[admlnx@linux-free ~]$ az acr repository list --name moneyhubrm9999 --output table
Result
-----
api-dimdim
api-transacoes
mysql-dimdim
[admlnx@linux-free ~]$
```

Criar um Registro de Container Privado usando a CLI do Azure

10. Depois de efetuar o Push das imagens, remova-as do seu ambiente do Docker

```
docker rmi moneyhubrm9999.azurecr.io/api-transacoes:v1
```

```
docker rmi moneyhubrm9999.azurecr.io/api-dimdim:v1
```

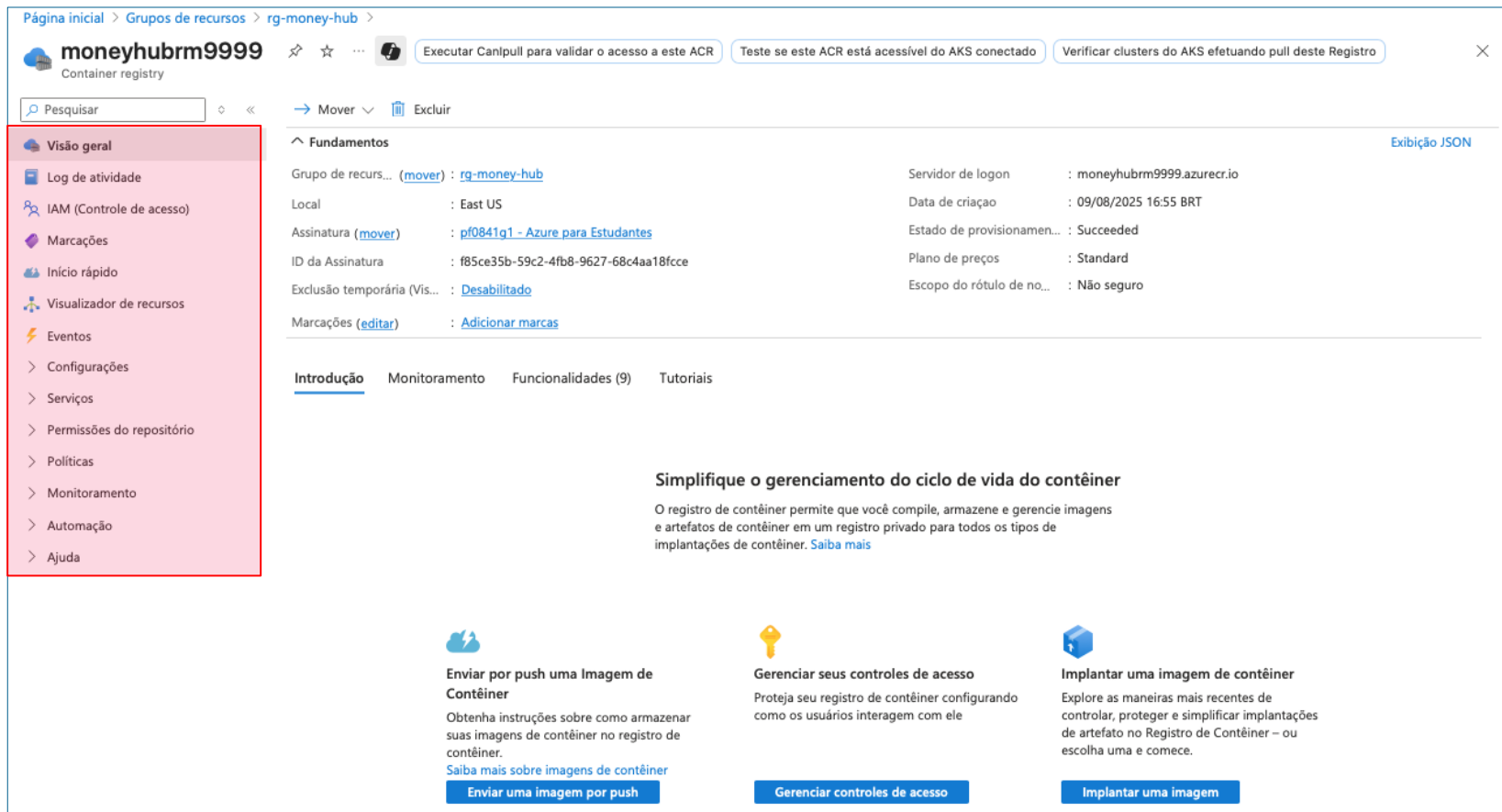
```
docker rmi moneyhubrm9999.azurecr.io/mysql-dimdim:v1
```

O comando **docker rmi** (sinônimo do comando *docker image rm*) não remove a imagem do repositório no Registro de Container do Azure



Criar um Registro de Container Privado usando a CLI do Azure

11. Analise juntamente com o Professor o Registro de Container criado e configurado



The screenshot shows the Azure Container Registry (ACR) portal for a registry named 'moneyhubrm9999'. The left sidebar is highlighted with a red box, showing the navigation menu with 'Visão geral' (Overview) selected. The main content area displays the 'Fundamentos' (Basics) tab, which includes a table of registry details and a section for 'Simplifique o gerenciamento do ciclo de vida do contêiner' (Simplify container lifecycle management).

Navigation Menu (Left Sidebar):

- Visão geral (Selected)
- Log de atividade
- IAM (Controle de acesso)
- Marcações
- Início rápido
- Visualizador de recursos
- Eventos
- Configurações
- Serviços
- Permissões do repositório
- Políticas
- Monitoramento
- Automação
- Ajuda

Registry Details (Fundamentos tab):

Propriedade	Valor
Grupo de recursos	rg-money-hub
Local	East US
Assinatura	pf0841g1 - Azure para Estudantes
ID da Assinatura	f85ce35b-59c2-4fb8-9627-68c4aa18fcce
Exclusão temporária (Visão)	Desabilitado
Marcações	Adicionar marcas
Servidor de login	moneyhubrm9999.azurecr.io
Data de criação	09/08/2025 16:55 BRT
Estado de provisionamento	Succeeded
Plano de preços	Standard
Escopo do rótulo de no...	Não seguro

Simplifique o gerenciamento do ciclo de vida do contêiner

O registro de contêiner permite que você compile, armazene e gerencie imagens e artefatos de contêiner em um registro privado para todos os tipos de implantações de contêiner. [Saiba mais](#)

Características Principais:

- Enviar por push uma Imagem de Contêiner:** Obtenha instruções sobre como armazenar suas imagens de contêiner no registro de contêiner. [Saiba mais sobre imagens de contêiner](#)
- Gerenciar seus controles de acesso:** Proteja seu registro de contêiner configurando como os usuários interagem com ele.
- Implantar uma imagem de contêiner:** Explore as maneiras mais recentes de controlar, proteger e simplificar implantações de artefato no Registro de Contêiner – ou escolha uma e comece.

Gerenciamento de Repositórios - Comandos essenciais

Listar os repositórios

```
az acr repository list --name moneyhubrm9999 --output table
```

Listar as tags (imagens) de um repositório

```
az acr repository show-tags --name moneyhubrm9999 --repository mysql-dimdim
```

Mostrar manifesto

```
az acr manifest list-metadata --registry moneyhubrm9999 --name api-dimdim
```

Limpar imagens antigas do ACR

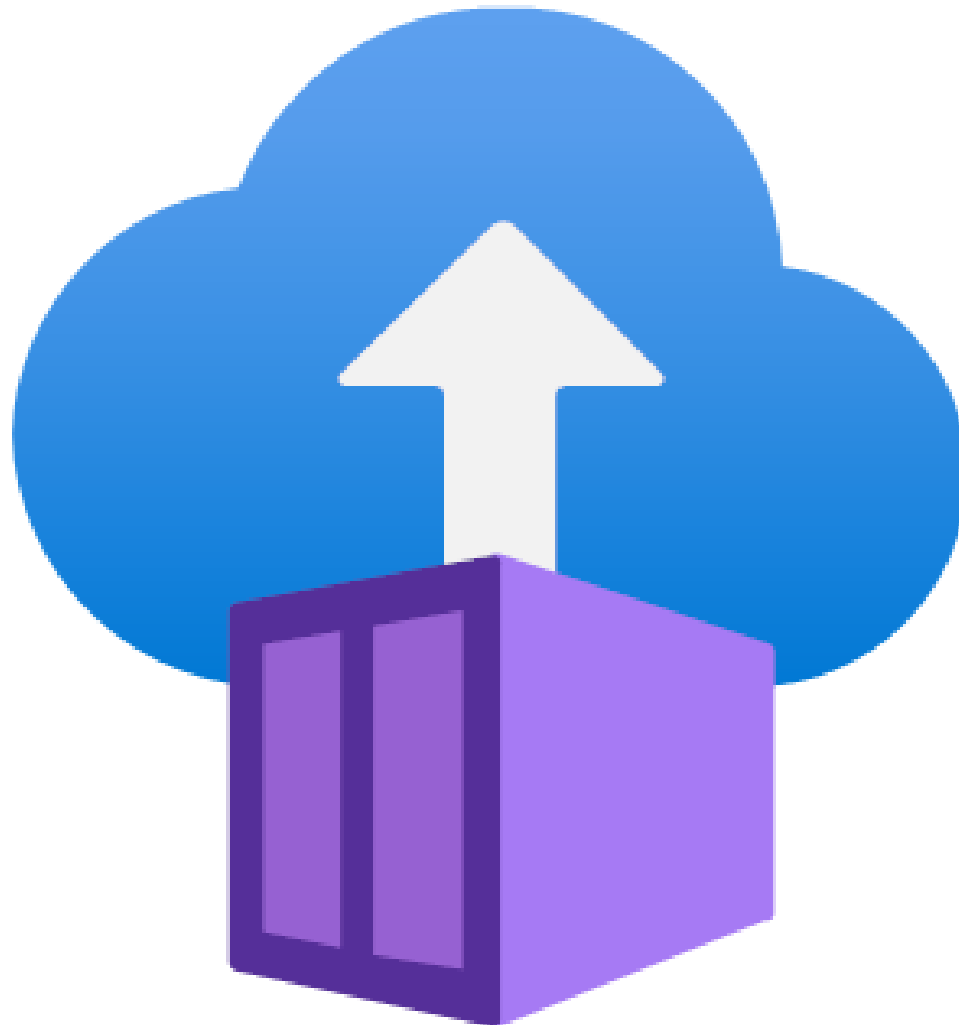
```
az acr purge --name nomeRepositorio --filter 'imagem:.*' --ago 7d --untagged
```

Detalhes de um repositório

```
az acr repository show --name moneyhubrm9999 --repository mysql-dimdim
```

Habilitar usuário administrador

```
az acr update --name <nome_do_registro> --admin-enabled true
```



Instâncias de Container



FIAP

O que são as Instâncias de Container do Azure?

Azure Container Instances (ACI)

- Utilizamos Containers para empacotar, implantar e gerenciar aplicativos em nuvem
- As Instâncias de Container do Azure oferecem uma maneira rápida e simples para executar um Container na nuvem, sem a necessidade de gerenciar Máquinas Virtuais nem adotar um serviço de nível superior
- É uma ótima solução para qualquer cenário que possa ser usado **em Containers isolados**, incluindo aplicativos simples, automação de tarefas e criação de trabalhos



Instâncias de Container



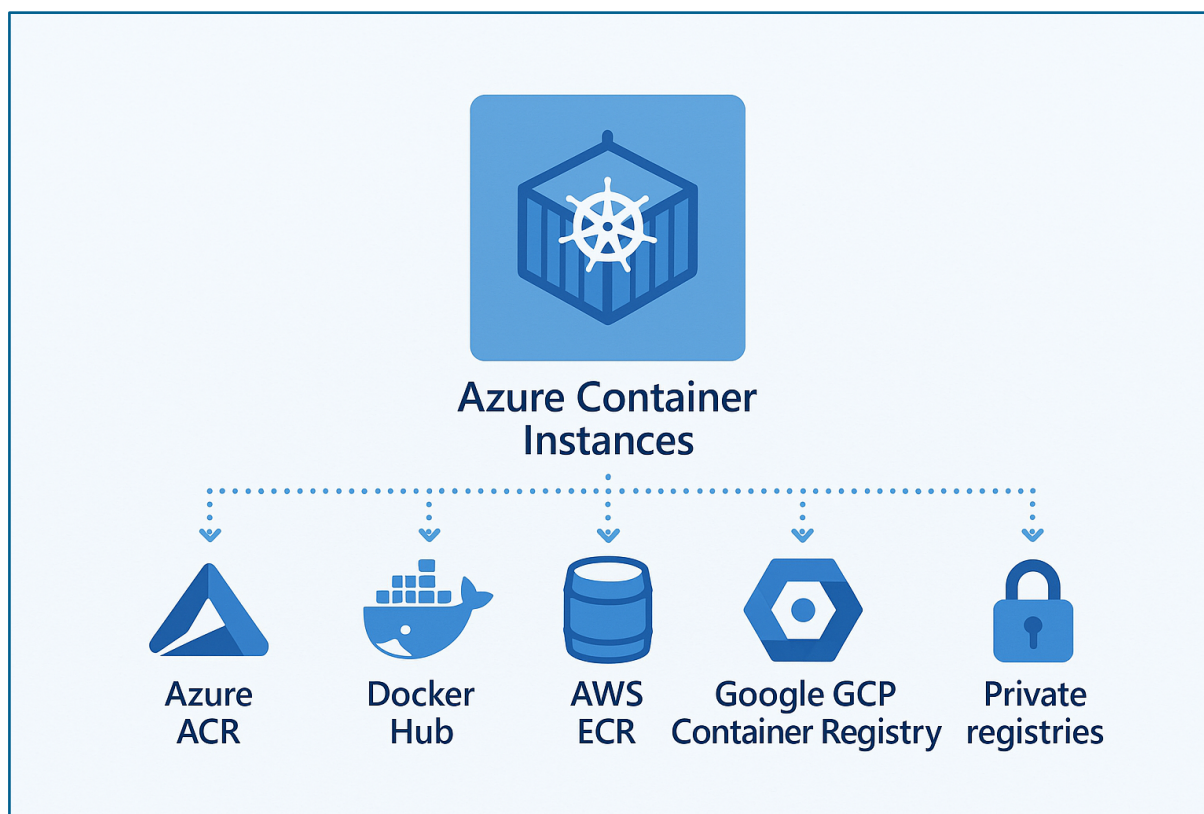
FIAP

Porque utilizar as Instâncias de Container do Azure?

- Você pode utilizar imagens de Containers do Linux/Windows do Docker Hub, de um registro de Container do Azure particular ou de outro registro do Docker baseado em nuvem
- As Instâncias de Container do Azure permitem expor seus grupos de Containers diretamente à Internet com um endereço IP e um FQDN (nome de domínio totalmente qualificado)
- Garantem que seu aplicativo fique tão isolado em um Container quanto ficaria em uma VM (Segurança em nível de Hipervisor)
- Fornecem uma utilização ideal ao permitirem especificações exatas de núcleos e memória da CPU
- Cobrança por segundo: gerar custos somente enquanto o container estiver sendo executado



O Azure Container Instances (ACI) oferece flexibilidade total para obter imagens de Containers de diversas fontes, permitindo que desenvolvedores e organizações utilizem seus repositórios preferidos sem restrições



Vamos começar a criar nossos Recursos em nuvem para dar suporte aos ACIs

- ✓ Localmente utilizamos uma *Rede Docker* para os Containers se comunicarem entre si. Em nuvem irão se comunicar através do IP Público
- ✓ Localmente utilizamos um *Volume Nomeado* para realizar a persistência de dados do Banco. Em nuvem vamos criar uma **Conta de Armazenamento**
- ✓ Localmente utilizamos *Variáveis de Ambiente*. Em nuvem vamos utilizar o **Azure Key Vault** para armazenar nossos dados sensíveis

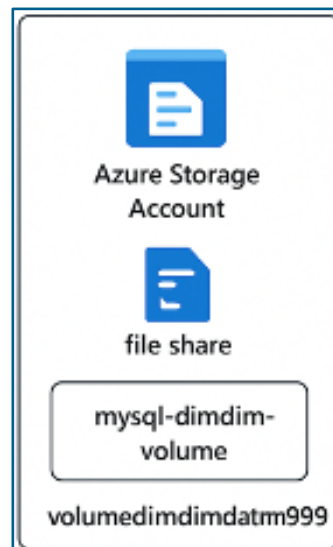
Analise e execute os Scripts na ordem para criar os Recursos necessários

01) Vamos criar nossa **Conta de Armazenamento**

ANTES DE EXECUTAR ALTERE O SCRIPT – SIGA ORIENTAÇÕES DO PROF

`chmod +x 01_store-account.sh`

`./01_store-account.sh > 01_store-account.log`



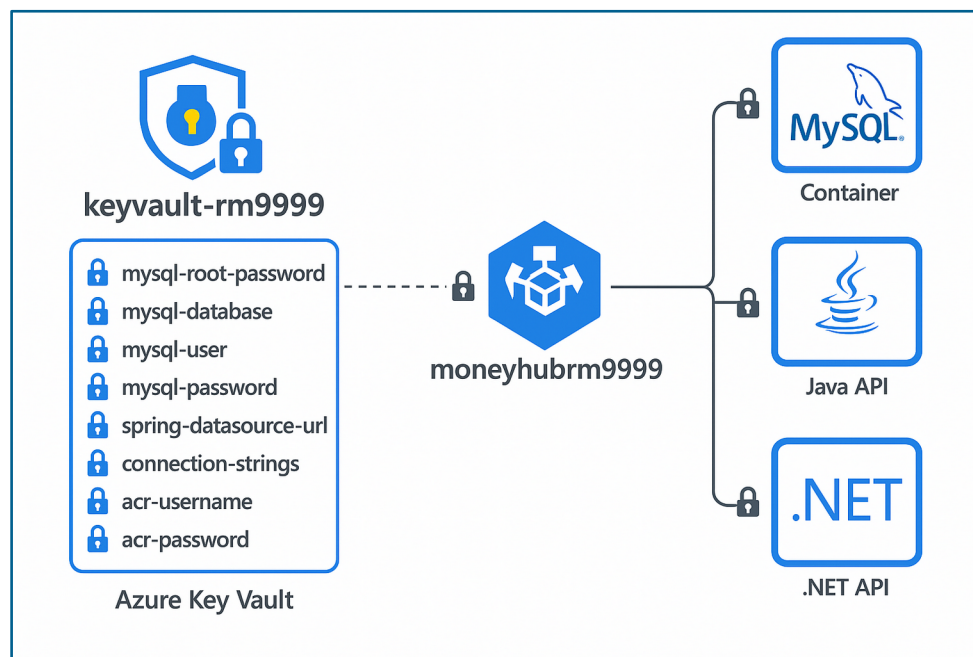
Verifique o resultado no Portal juntamente com o Professor

02) Vamos criar nosso Cofre de Chaves - **Key Vault**

ANTES DE EXECUTAR ALTERE O SCRIPT – SIGA ORIENTAÇÕES DO PROF

chmod +x 02_key-vault.sh

./02_key-vault.sh > 02_key-vault.log



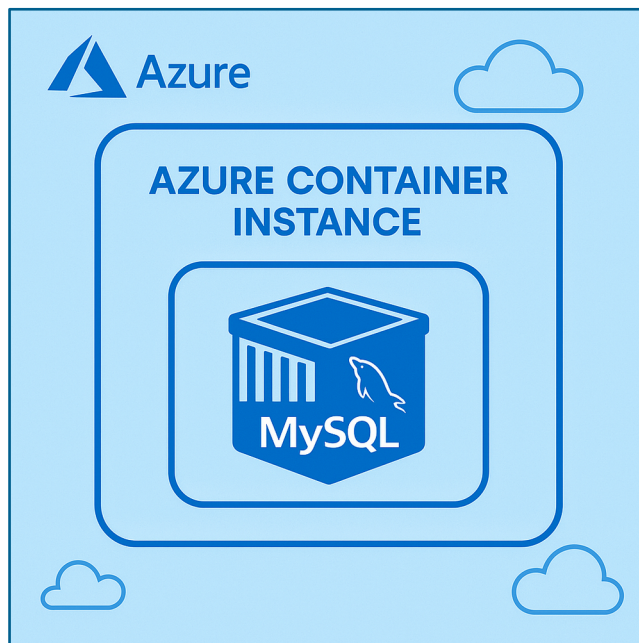
Verifique o resultado no Portal juntamente com o Professor

03) Todos os Recursos criados e configurados, vamos criar nosso ACI do MySQL

ANTES DE EXECUTAR ALTERE O SCRIPT – SIGA ORIENTAÇÕES DO PROF

`chmod +x 03_aci-mysql.sh`

`./03_aci-mysql.sh > 03_aci-mysql.log`



Verifique o resultado no Portal juntamente com o Professor

Instâncias de Container – Criação do Container MySQL

Execute o Container do MySQL que está em nuvem através da sua máquina

```
az container logs --resource-group rg-money-hub --name mysql-dimdim
```

```
az container exec --resource-group rg-money-hub --name mysql-dimdim --exec-command "mysql  
-uuser-dimdim -psenha-dimdim"
```

```
use db-dimdim;
```

```
select * from transacoes;
```

```
Menk — admlnx@linux-free:~ — ssh admlnx@104.41.52.221 — 73x14

[mysql> select * from transacoes;
+----+-----+-----+-----+
| id | descricao                | valor  | data_transacao |
+----+-----+-----+-----+
| 1  | Depósito bancário        | 1500.00 | 2025-06-01 10:00:00 |
| 2  | Transferência PIX        | -250.00 | 2025-06-03 14:30:00 |
| 3  | Pagamento de conta      | -180.50 | 2025-06-05 09:15:00 |
| 4  | Saque em caixa eletrônico | -300.00 | 2025-06-07 16:45:00 |
| 5  | Investimento em poupança | -500.00 | 2025-06-10 11:20:00 |
+----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql>
```

Instâncias de Container – Criação do Container MySQL

FLAP



O Banco MySQL está disponível na Nuvem

Setup New Connection

Connection Name: ACI - MySQL

Connection Method: Standard (TCP/IP)

Parameters SSL Advanced

Hostname: 20.242.215.9 Port: 3306

Username: user-dimdim

Password: Store in Keychain ... Clear

Default Schema:

Test Connection

Endereço IP (Public) : 20.242.215.9

Connect to MySQL Server

Please enter password for the following service:

Service: Mysql@20.242.215.9:3306

User: user-dimdim

Password:

Save password in keychain

Cancel OK

Successfully made the MySQL connection

Information related to this connection:

Host: 20.242.215.9
Port: 3306
User: user-dimdim
SSL: enabled with ECDHE-RSA-AES128-GCM-SHA256

A successful MySQL connection was made with the parameters defined for this connection.

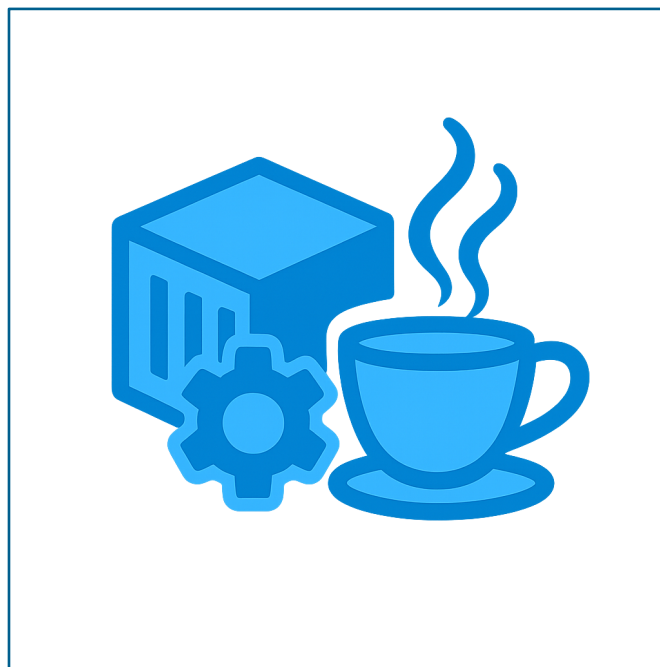
OK

04) Vamos criar nosso ACI da Api de Java

ANTES DE EXECUTAR ALTERE O SCRIPT – SIGA ORIENTAÇÕES DO PROF

```
chmod +x 04_aci-api-java.sh
```

```
./04_aci-api-java.sh > 04_aci-api-java.log
```



Verifique o resultado no Portal juntamente com o Professor

Instâncias de Container – Criação do Container Api Java



Execute o Container da API de Java que está em nuvem através da sua máquina

```
az container logs --resource-group rg-money-hub --name api-java
```

```
az container exec --resource-group rg-money-hub --name api-java --exec-command "/bin/bash"
```

```
cat /etc/os-release
```

```
curl -X GET http://localhost:8080/api/transacoes
```

```
Menk — adminx@linux-free:~ — ssh adminx@104.41.52.221 — 109x9
appuser@SandboxHost-638909673935576670:/app$
appuser@SandboxHost-638909673935576670:/app$ curl -X GET http://localhost:8080/api/transacoes
[{"id":5,"descricao":"Investimento em poupança","valor":-500.00,"dataTransacao":"2025-06-10T11:20:00"}, {"id":4,"descricao":"Saque em caixa eletrônico","valor":-300.00,"dataTransacao":"2025-06-07T16:45:00"}, {"id":3,"descricao":"Pagamento de conta","valor":-180.50,"dataTransacao":"2025-06-05T09:15:00"}, {"id":2,"descricao":"Transferência PIX","valor":-250.00,"dataTransacao":"2025-06-03T14:30:00"}, {"id":1,"descricao":"Depósito bancário","valor":1500.00,"dataTransacao":"2025-06-01T10:00:00"}]
appuser@SandboxHost-638909673935576670:/app$
appuser@SandboxHost-638909673935576670:/app$
```

Pode ocorrer uma falha no primeiro comando:

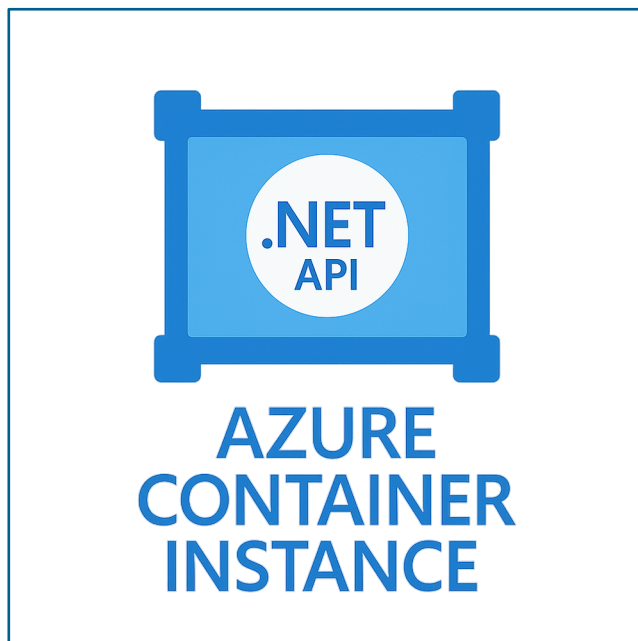
HikariPool-1 - Failed to validate connection com.mysql.cj.jdbc.ConnectionImpl@5432dca2 (No operations allowed after connection closed.). Possibly consider using a shorter maxLifetime value.

05) Vamos criar nosso ACI - Api de .NET

ANTES DE EXECUTAR ALTERE O SCRIPT – SIGA ORIENTAÇÕES DO PROF

```
chmod +x 05_aci-api-dotnet.sh
```

```
./05_aci-api-dotnet.sh > 05_aci-api-dotnet.log
```



Verifique o resultado no Portal juntamente com o Professor

Instâncias de Container – Criação do Container Api .NET

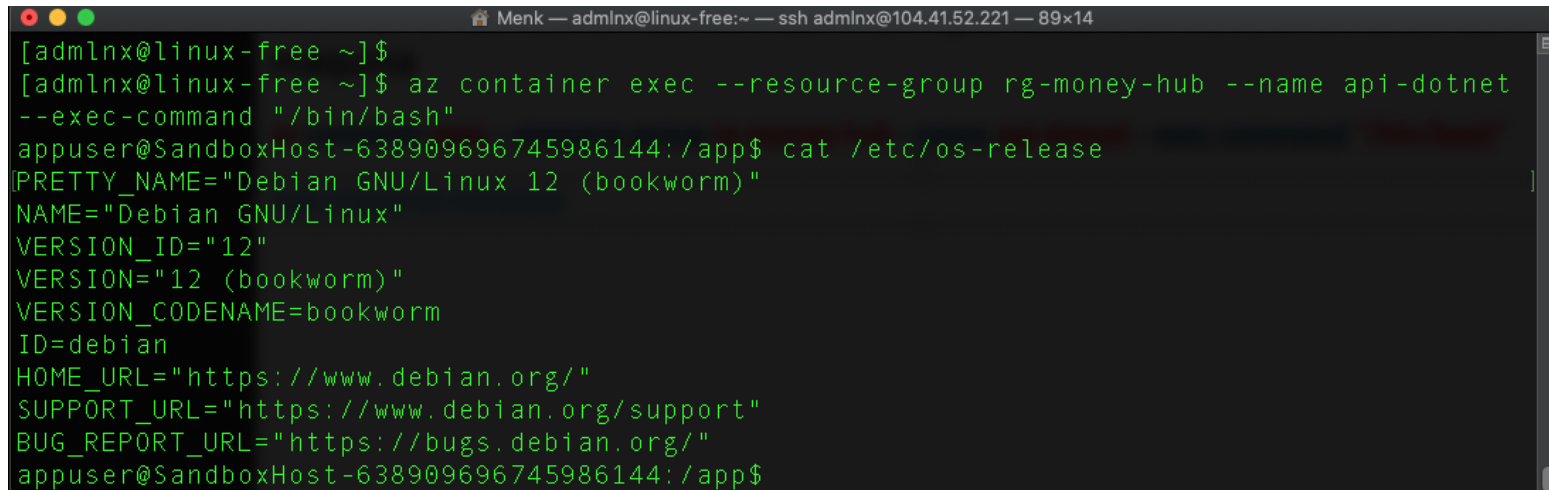
Execute o Container da API de .NET que está em nuvem através da sua máquina

```
az container logs --resource-group rg-money-hub --name api-dotnet
```

```
az container exec --resource-group rg-money-hub --name api-dotnet --exec-command "/bin/bash"
```

```
cat /etc/os-release
```

```
exit
```



```
Menk — adminx@linux-free:~ — ssh adminx@104.41.52.221 — 89x14
[adminx@linux-free ~]$
[adminx@linux-free ~]$ az container exec --resource-group rg-money-hub --name api-dotnet
--exec-command "/bin/bash"
appuser@SandboxHost-638909696745986144:/app$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
NAME="Debian GNU/Linux"
VERSION_ID="12"
VERSION="12 (bookworm)"
VERSION_CODENAME=bookworm
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
appuser@SandboxHost-638909696745986144:/app$
```

Execute no terminal de sua máquina

```
fqndotnet=$(az container show --resource-group rg-money-hub --name api-dotnet --query ipAddress.fqdn --output tsv)
```

```
curl -X GET http://$fqndotnet:8080/api/transacoes
```

API Java

```
az container exec --resource-group rg-money-hub --name api-java --exec-command "/bin/bash"
```

```
curl -X POST http://localhost:8080/api/transacoes \  
-H "Content-Type: application/json" \  
-d '{  
  "descricao": "Compra no supermercado",  
  "valor": 150.75  
'
```

API .NET

Execute no terminal de sua máquina

```
curl -X PUT http://$fqdn.net:8080/api/transacoes/6 \  
-H "Content-Type: application/json" \  
-d '{  
  "id": 6,  
  "descricao": "Compra no supermercado - ALTERADO",  
  
  "valor": 150.76,  
  "dataTransacao": "2024-06-18T00:00:00"  
'
```

API Java

```
az container exec --resource-group rg-money-hub --name api-java --exec-command "/bin/bash"
```

```
curl -X DELETE http://localhost:8080/api/transacoes/6
```

API .NET

Execute no terminal de sua máquina

```
curl -X GET http://$fqdn_dotnet:8080/api/transacoes
```

Instâncias de Container – Comandos essenciais

Visualizar logs do container

```
az container logs --resource-group rg-money-hub --name api-java
```

Logs com follow (streaming)

```
az container logs --resource-group rg-money-hub \  
  --name api-java \  
  --follow
```

Executar comando interativo

```
az container exec --resource-group rg-money-hub \  
  --name api-java \  
  --exec-command "/bin/bash"
```

Executar comando específico

```
az container exec --resource-group rg-money-hub \  
  --name api-java \  
  --exec-command "ls -la"
```

Verificar processos

```
az container exec --resource-group rg-money-hub \  
  --name api-java \  
  --exec-command "ps aux"
```

Deletar containers específicos

```
az container delete --resource-group rg-dimdim-containers --name aci-dimdim-api --yes
```

Copyright © 2026 Prof. João Menk

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).